

Doxa and Logos: Formalizing the Functions of a Non-Formalizable Source in Neuro-Symbolic Systems

Vladimir Lapshin*

Abstract

Formal systems have a fundamental limit: Gödel’s and Turing’s theorems describe the boundaries of closed systems, not of human thinking, which constantly expands and revises its formalisms. We propose a distinction between *doxa* — an unformalizable source of lived experience, the cultural field, and bodily knowledge — and *logos* — rigorous, verifiable knowledge. We claim that although the content of doxa is unformalizable, the functions of its interaction with logos can be described formally. We introduce a family of interaction operators and distinguish three roles within it: *external* (doxastic) operators supply a proposal not derivable from the theory; *internal* (logotic) operators compute the system’s reaction; *protocol* operators record, classify, and control the exchange, making it checkable. We show that this distinction has rigorous formal analogues in modern logic — doxastic logic, non-monotonic logics, abduction — but none of them formalizes doxa as such. Finally, we describe a working system, Ankyra, in which a large language model plays the role of doxa and a symbolic engine plays the role of logos; the principle “the LLM proposes, the engine decides” ensures transparency: nothing enters the theory without a verbatim quotation or an explicit hypothesis tag, and the strength of an answer never exceeds the strength of its premises.

1 Introduction

Formal systems have a fundamental limit. Gödel’s incompleteness theorems showed that any sufficiently rich consistent system contains statements it can neither prove nor disprove [1]. Turing’s halting problem showed that no general algorithm exists that, for any program and input, would determine whether it halts [2]. These results are usually interpreted as absolute limits of cognition.

We propose a different interpretation. These are the limits of *static* systems — closed sets of axioms and inference rules. But human thinking is not a static system. It does not begin with a fixed set of axioms and derive consequences to exhaustion. It constantly expands, revises, and augments its formalisms, relying on what the ancient tradition called *doxa*.

The distinction between doxa and logos has a long history. Already in Xenophanes (6th c. BCE), *doxa* ($\delta\acute{o}\xi\alpha$) denotes knowledge with *indeterminate truth value* — as opposed to certain knowledge. Parmenides (5th c. BCE) contrasts $\delta\acute{o}\xi\alpha$ with *truth* ($\alpha\lambda\acute{\eta}\theta\epsilon\iota\alpha$): doxa is the “deceptive

*merfill@yandex.ru

order” of the sensory world, whereas truth is accessible only to *logos* — pure thought that does not trust the evidence of eyes and ears [3]. It was Parmenides, as historians of philosophy note, who first drew the distinction between the data of the senses and the data of logos and asserted that *only the latter is real and true*.

Plato radicalized this distinction. For him, $\delta\acute{o}\xi\alpha$ is cognition of *what appears*, whereas *episteme* ($\epsilon\pi\iota\sigma\tau\acute{\eta}\mu\eta$) is cognition of *what is*. Doxa may be true or false, but it always remains *unstable, finite, contingent* — it “is expressed by relation to an object, not by justification of truth” [4]. Episteme, by contrast, is knowledge *accompanied by logos*, that is, by explanation, justification, *rigorous reasoning*. Aristotle then refined this: doxa is *opinion*, which may be true but lacks *necessary* character; episteme is knowledge of *the necessary*, obtained through *demonstration* [5].

Thus, in the ancient tradition a stable distinction emerged: *doxa* is opinion, experience, probabilistic judgment rooted in the sensory and cultural; *logos* (or *episteme*) is rigorous, justified, verifiable knowledge. Doxa is *wider*, but *less reliable*. Logos is *narrower*, but *guaranteed*.

We use these terms in their original sense, with one principled clarification. *Doxa is non-formalizable*. It is not “under-formalized knowledge,” not “incomplete logos,” not “raw material for formalization.” It is *a different type of knowledge*. Logos is not “improved doxa.” It is *a product of a different coordinate system*. Doxa is *lived experience, collective non-formalizable knowledge, the cultural field, embodiment, historicity*. It is *wider* than logos but *does not reduce* to it. It *does not prove* — it *proposes*. It does not *guarantee* — it *suggests probabilistically*. And this is *not a deficiency*. This is its *nature*. This is the *foundation* of the distinction.

The attempt to formalize everything is *excessive formalization*, a relic of an era when mathematics seemed the only path to rigorous knowledge and everything non-formalizable was considered inferior. In reality, human thinking always relies on *both* sources: on doxa as an inexhaustible background of lived experience, and on logos as an instrument of verification and guarantee. Doxa is *not cancelled* by logos. It *nourishes* it.

The key thesis of our work is the following. Although the *content* of doxa is fundamentally non-formalizable, the *functions* of doxa — how it is used to expand formal systems — can be *described formally*. We introduce a family of operators of the doxa–logos interaction and distinguish three roles within it: *external* (doxastic) operators, which supply a proposal not derivable from the theory; *internal* (logotic) operators, by which the formal system computes its reaction to that proposal; and *protocol* operators, which record, classify and control the exchange. Only the external operators take the current formal theory and return an *extension* marked as a *proposal*, not as a *derivation*; the fate of that proposal is decided by logos, which *does not produce* it itself.

This approach does not attempt to “overcome” Gödel’s theorems. It accepts them as a description of the limits of *closed* systems and shows how an *open* system can develop, using doxa as an external source. The relation to the notion of an *oracle* is structural: doxa is not part of the machine, but the machine can consult it [6]. Doxa is *external* to the formal system; this does not require it to be non-computable, a point we return to in Sections 3 and 5.

The paper is structured as follows. In Section 2 we examine the theoretical foundations: the

distinction between doxa and logos in the ancient tradition and in modern logic, a survey of doxastic logic, non-monotonic logics, abduction, and dynamic logics of belief. In Section 3 we introduce the operator family of the doxa–logos interaction and its formal properties. In Section 4 we describe a concrete implementation of this model. In Section 5 we discuss the limits of the approach, its relation to hypercomputation, and practical implications for explainable AI. Section 6 concludes.

2 Theoretical Foundations

In Section 1 we introduced the distinction between doxa and logos and showed its ancient origins. Here we will not repeat the history. Our task is to show that this distinction has *rigorous formal analogues* in modern logic and that these analogues are *relevant* to our model.

We do not claim that these formalisms “solve” the problem of doxa. We claim that they *describe particular aspects* of how doxa functions. And these aspects *can be used* to build a working system.

Doxastic logic

The term “doxastic logic” was introduced by *Hintikka* in 1962 for the formal modeling of *opinions and beliefs* (δόξα) of an agent, as distinct from knowledge (ἐπιστήμη) [7]. The basic operator B_α reads as “Alpha believes that...”. Doxastic logic deals with *synchronic* principles — how beliefs are related to one another at a given moment.

But it is *silent* about how beliefs change over time. This limitation was recognized, and the response to it was *dynamic doxastic logic* (DDL), introduced by *Krister Segerberg* in the 1990s [8]. DDL adds to static doxastic logic *modal operators of belief change*: $[*p]q$ reads as “ q holds after revision of beliefs by p ” [9].

Here an important clarification is needed. *Doxa does not change*. Doxa is an inexhaustible source of lived experience, the cultural field, bodily knowledge. It is *primary* and *invariant* in the sense that it is not an object of revision. What changes is the *formal system* — logos. The beliefs that DDL speaks of are a *formalized slice* of doxa, fixed in the language of logic. DDL describes the *protocol of revision* of this slice when new information arrives. But doxa itself remains *beyond* revision. It is the *source*, not the *object*.

Thus DDL is a formal analogue not of doxa as such, but of the *operator of revision of the formal system* under the influence of doxa. This is precisely what we call a *function of doxa*.

Non-monotonic logics

Classical logic is *monotonic*: if $\Gamma \vdash \phi$, then $\Gamma \cup \Delta \vdash \phi$. Adding new axioms *does not cancel* previously derived consequences. But a formal system interacting with doxa works differently. New experience may *cancel* old conclusions. If I know that Tweety is a bird, I assume he can fly. If I later learn that Tweety is a penguin, I *cancel* the conclusion “Tweety flies,” although it previously followed from “Tweety is a bird.”

Non-monotonic logics are formal systems that model precisely this *defeasibility* of conclusions. Classical approaches include:

- **Default Logic** by *Reiter*: rules of the form “if A and there is no information about exceptions, then B ,” which can be defeated when an exception appears [10].
- **Circumscription** by *McCarthy*: minimization of “abnormal” situations, allowing default conclusions [11].
- **Autoepistemic logic** by *Moore*: modeling an agent’s beliefs about its own beliefs [12].

All these formalisms acknowledge that *the conclusions of a formal system are not final*. They are *defeasible* when new information from doxa arrives. And this is not a “deficiency.” It is a *property* of a system that is *open* to interaction with doxa. Non-monotonic logics formalize this property but *do not eliminate* it. They show how a system can *work* with defeasible conclusions without losing coherence.

Abduction

If non-monotonic logics describe the *cancellation* of conclusions, then *abduction* describes the *generation* of new hypotheses. The term was introduced by *Charles Sanders Peirce* (1839–1914), who distinguished three types of inference: deduction, induction, and *abduction* — “the creative generation of explanatory hypotheses” [13].

Peirce characterized abduction as reasoning “from consequence to cause.” If we observe a fact Q , and know that $P \rightarrow Q$, abduction allows us to *suppose* P as a *possible explanation* of Q . This is not deduction (where P *necessarily* follows) nor induction (where P *probably* follows). It is the *generation of a hypothesis* that *explains* the observation [14].

In contemporary philosophy of science, abduction is often identified with “*inference to the best explanation*” (IBE). However, as researchers note, *the identification of abduction with IBE is not rigorous*. Peirce distinguished the *generation* of a hypothesis (abduction in the narrow sense) from the *selection* of the best one (IBE). For our model, it is precisely *generation* that matters: doxa *proposes* hypotheses, and the formal system *verifies* them [15].

Abduction is *formalizable* — but only on condition that we formalize the entire “pattern” of abductive inference, including the rules of hypothesis generation and justification. There exist formal systems in which abduction is added to propositional logic as an *external extension operator* [22]. This is precisely what we call the *operator of hypothesis generation*.

What remains beyond formalization

We have surveyed three formal approaches: doxastic logic (including DDL), non-monotonic logics, abduction. Each *captures* one aspect of the interaction between doxa and a formal system. But *none of them formalizes doxa as such*.

Doxastic logic formalizes the *structure* of beliefs as a formalized slice of doxa. DDL — the *protocol* of revision of this slice. Non-monotonic logics — the *defeasibility* of conclusions when new

information from doxa arrives. Abduction — the *generation* of hypotheses that doxa proposes for extending the system. But the *content* of doxa — its *embodiment, historicity, contextuality* — remains *beyond* these formalisms.

And this is not a “deficiency” of these systems. It is a *boundary* they themselves acknowledge. Non-monotonic logics, for example, are *computationally more complex* than classical logic [16], and for many of them the problem of deciding whether a goal is a consequence is *undecidable* or of very high complexity [23]. This means that *no general algorithm exists* that could, for any theory and goal, determine whether the goal is a consequence. We can build a *procedure*, but we cannot guarantee that it terminates.

Abduction, in turn, generates a *multitude* of hypotheses, and choosing among them is a *separate task* with no formal criterion of “truth.” Propositional abduction is a Σ_2^P -complete problem, that is, *harder* than NP and coNP [18], and logic-based abduction is computationally hard in general [17].

This is precisely why our model *does not attempt* to formalize doxa. It *uses* formal systems to describe the *functions* of doxa, leaving the *content* of doxa beyond formalization. This is *not a defeat*. It is an *acknowledgment of the boundary* and the *construction of an interface* to it. It is worth stressing that non-formalizability is not the same as non-computability: doxa is not a theory in L , but the carrier through which it is consulted need not be a non-computable oracle.

Connection to our model

We claim that the *functions* of doxa — how it is used to expand formal systems — can be *described formally*. In the family of Section 3 each formal approach considered above corresponds to one operator, and the operator belongs to one of the three roles:

- **Doxastic logic** $\rightarrow$ operator of *fixation* (what the agent “holds” as beliefs). This is a *protocol* function of the environment, realized by the journal.
- **Dynamic doxastic logic** $\rightarrow$ operator of *revision of the formal system*. This is *internal*: *logos* computes how the theory changes when new information from doxa arrives.
- **Non-monotonic logics** $\rightarrow$ operator of *cancellation*. This is *internal*: *logos* recomputes what follows when exceptions obtained from doxa appear.
- **Abduction** $\rightarrow$ operator of *hypothesis generation*. This is *external*: doxa supplies a hypothesis that the formal system verifies.

This is *not* an exhaustive list. We will return to it in Section 3, where we introduce the *family of operators of the doxa–logos interaction*. But it is already clear: the distinction between doxa and logos is *not* a philosophical abstraction. It has *rigorous formal analogues* in modern logic. And these analogues *can be used* to build a working system.

3 The Operator Family of the Doxa–Logos Interaction

In Section 2 we showed that the distinction between doxa and logos has formal analogues in modern logic. These analogues describe particular aspects of the interaction between doxa and a formal system. Now we turn to *synthesis*. Our task is to describe the *functions* through which doxa and logos interact — not the content of doxa, which is non-formalizable, but the *protocol* of its use. The central point of this section is that this protocol is *asymmetric*: doxa contributes the content of proposals, while everything that decides, records and controls their fate belongs to logos and to the environment of the interaction.

Basic definitions and the role criterion

Let $F = (A, R, L)$ be a formal system with axioms A , inference rules R and language L , and let $\text{Cn}(F)$ be its consequence set. Let D be doxa: an inexhaustible field of lived experience, the cultural field, bodily knowledge, historicity. D is not part of F , and it is not a theory in L .

Two clarifications are needed before the operators are defined. First, *non-formalizability is not non-computability*. Doxa is non-formalizable in the sense that it is not a system whose content can be captured by axioms and rules of L ; this does not mean that the *carrier* through which it is consulted must be a non-computable oracle. What matters for the model is that the increment doxa supplies is *external* to F . Second, an operator may extend the *theory* (new axioms, rules or objects) or the *language* itself; a change of L changes what counts as a formula, and the two kinds of extension must not be conflated.

Let an operator be a transition $o : F \rightarrow F'$, and let $\Delta = F' \setminus F$ be its increment. The classification of the operators is by *role*, not by position: protocol functions, too, lie outside F , but they are not a source of content.

- An **external** (doxastic) operator introduces an increment that is not derivable from F , i.e. $\Delta \not\subseteq \text{Cn}(F)$; its content originates in doxa.
- An **internal** (logotic) operator computes F' from F and from input already supplied; every element of Δ is determined by F and that input.
- A **protocol** operator neither introduces content nor computes in F ; it makes the exchange well-defined, deterministic and auditable.

Class	Members	Role
External (doxastic)	$o_{\text{abd}}, o_{\text{anal}}, o_{\text{int}}$	supply a proposal not derivable from F
Internal (lo- gotic)	$o_{\text{rev}}, o_{\text{def}}, o_{\text{spec}}, o_{\text{ctx}}$	compute the reaction of logos within F
Protocol (en- vironment)	journal/fixation, classifica- tion, declaration, ground- ing, waves	make the exchange well-defined and auditable

External operators (doxastic)

Operator of generation (o_{abd}). Takes F and an observation Q and returns F' containing a *hypothesis* P that explains Q . Formally $o_{\text{abd}} : F \times Q \rightarrow F'$ with $F' = F \cup \{P\}$ and $P \notin \text{Cn}(F)$. This is the formal analogue of abduction. The hypothesis is *proposed*, not derived; logos checks it for compatibility with F and for explanatory power [22].

Operator of analogy (o_{anal}). Takes a familiar domain F_1 and a new domain F_2 and returns F'_2 in which the structure of F_1 is transferred to F_2 . The choice of the structural mapping is external; logos only verifies whether it is preserved.

Operator of intuition (o_{int}). Takes F and a goal ϕ and returns an answer ϕ' that does not follow from F . This is the operator closest to an oracle. Doxa *gives* the answer; logos verifies it if it can, and otherwise defers it as a hypothesis. What is oracle-like here is the *source* of the answer, not the carrier, which may well be computable.

Internal operators (logotic)

These operators are often mistaken for functions of doxa because they are triggered by doxa's input. In fact they are computed by logos; the input is only their argument.

Operator of revision (o_{rev}). Given F and new information p (supplied externally), returns F' in which beliefs are revised in light of p . Formally $o_{\text{rev}} : F \times P \rightarrow F'$. This is the formal analogue of AGM revision [21] and of dynamic doxastic logic (DDL). Doxa supplies p ; logos decides how p affects F .

Operator of cancellation (o_{def}). Given F and an exception e (supplied externally), returns F' in which previously derived consequences are cancelled. Formally $o_{\text{def}} : F \times E \rightarrow F'$. This is the formal analogue of non-monotonic logics. Cancellation is not deletion of axioms: what was derivable ceases to be derivable under the new configuration.

Operator of specificity (o_{spec}). Given F and two conflicting rules r_1, r_2 , returns F' in which the more specific one is selected. Formally $o_{\text{spec}} : F \times \{r_1, r_2\} \rightarrow F'$. Specificity is a relation *computed by logos* over the hierarchy of F ; doxa does not supply it. This is the formal analogue of specificity-based preference in non-monotonic logics.

Operator of context application (o_{ctx}). Given F and a declared context Γ , returns F' in which the meanings of terms are redefined in accordance with Γ . This is the formal analogue of situation logic. The context is *declared* by the protocol (see below); its *application* is internal.

Protocol operators (environment)

These functions belong neither to doxa nor to logos. They introduce no content and compute nothing in F , yet without them the interaction cannot be carried out correctly.

Journal and fixation (o_{fix}). Each step is recorded in a journal: the identifier of the operator, the wave, the rationale, the source, and the result (accepted, rejected or deferred). The journal is the protocol of the interaction and is part neither of doxa nor of logos. Fixation here means

selecting the formalized slice of doxa that is being consulted; it is a bookkeeping act of the environment, not a function of doxa.

Deterministic classification. Every proposal is classified by fixed rules (see the role of logos below), independently of any “mood”; this is what keeps the exchange reproducible.

Context and fragment declaration. A context Γ or a required fragment of a logic is *declared* as a parameter of the run; its application is internal (see o_{ctx} above).

Grounding checks. Verbatim-quote checks and other structural witnesses certify that a proposal is anchored in its source.

Wave protocol. The rule “at most one proposal per consulting act” makes each act verifiable and leaves a trace of where and why the theory changed.

Properties of the family

The external operators do not commute, and the order of their application is essential. Suppose F contains `is_a(tweety, bird)` and the default `bird  $\Rightarrow$  fly`, and suppose doxa may propose `is_a(tweety, penguin)`, after which logos has a more specific default `penguin  $\Rightarrow$   $\neg$ fly`. Applying o_{abd} first and o_{spec} second selects the penguin default and cancels `fly`. Applying o_{spec} first finds no competing rule and concludes `fly`; the later o_{abd} then does not select but forces a *revision*. The same operators in two orders yield different theories: the process is historical, and each step changes the space of possible next steps.

The family is not reducible to a single operator. The expression $o_1 \cup o_2$ is not an operator: because the effect of o_2 depends on whether o_1 has already been applied, there is no function of the initial F alone that captures the pair.

External operators do not guarantee a result. o_{abd} may propose a contradictory hypothesis, o_{anal} an inappropriate mapping, o_{int} an erroneous answer. This is not a defect but the nature of doxa: it proposes, it does not guarantee. Internal operators, by contrast, are deterministic functions of F and their input; protocol operators guarantee determinism and auditability, not truth.

The process of reasoning

Reasoning is a *sequence* of extensions:

$$F_0 \xrightarrow{o_1} F_1 \xrightarrow{o_2} F_2 \xrightarrow{o_3} \dots$$

where each step is an *act* of consulting doxa, labelled by the operator that mediates it; the recording, classification and application of each act are performed by the protocol. Not a “computation” but a *consultation* of the source.

Each step is *recorded* in a *journal*: the identifier of the operator, the wave, the rationale, the source, and the result (accepted / rejected / deferred). The journal is the *protocol* of interaction

between doxa and logos. It is *not part* of either doxa or logos. It is the *interface* — a protocol operator, not a function of doxa.

The role of logos

Logos is *not passive*. It *classifies* each proposal of doxa:

- **derivable** — the proposal already follows from F . Accepted as *narrative*.
- **cited** — the proposal is grounded in a *quotation* from the text. Added to the axioms.
- **hypothesis** — the proposal is new. Recorded in the journal with the mark H .
- **rejected** — the proposal fails verification. Discarded with a reason code.

The classification is a protocol function; the verdict it produces belongs to logos. It is *deterministic*: it depends on the *structure* of F and the *content* of the proposal, not on the “mood” of logos. This is the *guarantee* that logos does not “yield” to doxa. It *accepts* only what it *can* accept. And it *honestly* says when it *cannot*.

Limits of the model

The model *does not claim* to formalize doxa. It *does not say* what doxa “really is.” It *does not assert* that doxa “reduces” to operators. It *describes* only the *functions* through which doxa *interacts* with logos.

The model *does not guarantee* completeness. The list of operators is *open*. There may be operators we have *not identified*. There may be aspects of doxa that are *not expressed* through operators. The model *does not claim* otherwise.

The model *does not solve* the problem of incompleteness. It *accepts* it as a *boundary* of logos and shows how an *open* system can *develop*, using doxa as an *external source*. This is *not an overcoming* of Gödel. It is *life* with Gödel. Not “after” him. But *together* with him.

4 Implementation: Ankyra

In Section 3 we introduced the operator family of the doxa–logos interaction. Here we show that the model is not a theoretical construct: it is implemented in a working system, the hybrid neuro-symbolic reasoning engine Ankyra [19]. We describe how each of the three roles is realized — the external operators, the internal operators and the protocol — and note that a full description of the architecture is the subject of a separate work.

Doxa is the LLM

The key claim we wish to make explicit: *doxa in Ankyra is a large language model*. Not an “analogy.” Not a “metaphor.” Not a “philosophical illustration.” The LLM *is* the carrier through which doxa is consulted in machine form.

From the preceding exposition, the reader might have formed the impression that doxa is something mystical and inexpressible, accessible only to human consciousness. We claim the opposite. Doxa is a *field of knowledge* accessible through language. The LLM is trained on texts, i.e., on language, and language is not merely a means of transmitting information. It is a *world-picture* — a lens that sets the grid of perception. The LLM is a hypertrophied version of this lens. It is trained on texts that already contain knowledge about the world, and it reconstructs reality through the patterns of language. It is a snapshot of snapshots — language about language, and precisely for this reason the LLM makes it possible to regard it as doxa: it *does not possess truth*, but it is *full of life*. The LLM as doxa in compressed form: the lived experience of humanity compressed into matrices of weights; the cultural field accessible through prediction of the next token; historicity frozen in probabilities.

It is important not to over-read the oracle analogy. The LLM is a computable function of its input, and nothing in the model requires a non-computable source. What makes it a doxa carrier is not a lack of computability but its *externality* to the formal theory: the increment it proposes is not derivable from the axioms and rules in L . Doxa is non-formalizable here in the precise sense of not being a theory in the language of logos — not in the sense of being beyond computation.

The principle: the LLM proposes, the engine decides

The core of Ankyra is the distinction between two participants. The LLM plays the role of doxa: it reads the text, decomposes it into structure, and in the reasoning cycle proposes additional steps. The symbolic engine plays the role of logos: it turns the structure into a formal theory, closes it under inference, matches it against the question, and certifies what actually follows.

The boundary is absolute. The LLM never possesses truth. Nothing enters the theory that the engine cannot justify either by a verbatim quotation from the text or by an explicit hypothesis mark.

The proposal contract

Every statement the LLM wishes to add is deterministically classified into one of four categories: **derivable** (already follows from the theory), **cited** (supported by a verbatim quotation), **hypothesis** (knowledge not present in the text), **rejected** (fails verification). This is a direct implementation of the operator of hypothesis generation (o_{abd}): the LLM proposes, the engine classifies, nothing is accepted without grounds. The distinction between **cited** and **hypothesis** makes the answers honest: a conclusion based on quotations is **proven**, a conclusion based on assumptions is **proven_under(H)** with a listing of those assumptions. An answer is never stronger than the grounds behind it.

The reasoning cycle

A task passes through four phases: *phase 0* — decomposition (the LLM decomposes the text into structure), *phase 1* — solving (the engine closes the theory), *phase 2* — the hint cycle

(the LLM proposes one step at a time if the theory does not solve the question), *phase 3* — explanation (the chain is assembled mechanically from the proof history).

The hint cycle (phase 2) is organized into *waves*. A wave is one complete act of consulting doxa: the engine checks the current state of the theory and formulates the gaps, the LLM receives a hint with these gaps and proposes *exactly one* typed proposal, the engine classifies this proposal and applies or discards it, after which the wave is recorded in the journal and the cycle begins anew. The constraint “one proposal per wave” is principled: it makes each consultation of doxa verifiable and leaves a trace by which one can reconstruct at which step and why the answer changed.

The hint cycle implements the process of consulting doxa from Section 3: each wave is one act $F_i \xrightarrow{o} F_{i+1}$, mediated by an *external* operator, while recording, classification and application are performed by the protocol.

The journal

Each step is recorded in the journal: the identifier of the operator, the wave, the rationale, the source, the result. The journal is the protocol of interaction between doxa and logos, not part of either. It realizes the *protocol* role of the family (journal and fixation): it records each act of consulting doxa and makes the process verifiable and reproducible. It is not a function of doxa.

Monotonic base and moving answer

The base — axioms, rules, hypotheses — only grows: premises are never deleted, the goal is never weakened. But the answer may change as new information arrives. Each change is recorded as a **Revision** with an indication of the wave and the assumptions involved. This is an *internal* operator of the family (revision, o_{rev}): doxa supplies the information, logos computes how it affects the theory, and the decision is recorded, not hidden.

Engines and the declared fragment

Ankyra is not a monolithic solver but an orchestrator of formalizations and decision procedures. Different kinds of tasks require different logics: Horn clauses (L0), stratified negation (L1), disjunction and quantifiers (L2), finite-domain constraints (L3), arithmetic (L4), defaults with specificity (D). These engines are *internal* operators of the family (logos-side): L1 and D realize cancellation (o_{def}) and specificity (o_{spec}) respectively, and L2 realizes case analysis within the decision procedure. None of them is a source of content.

The choice of procedure is carried out through the *contract of the declared fragment*: the engine examines the constructed theory and query and derives the set of required features. If the fragment fits within the available capabilities, the corresponding procedure is chosen; if not, the result is the named gap **out_of_fragment**, rather than a silent downgrade to a weaker logic. The declaration of the fragment is a *protocol* function; its application is internal, and the system honestly reports a mismatch instead of guessing.

Honesty by construction

The value of the system rests on guarantees provided mechanically: there are no facts in the theory belonging to the model; the question is never a source; there is no natural-language parsing in deterministic code; having assumed P , one cannot refute $\neg P$; an answer that used assumptions is always marked `proven_under(H)`. This is the implementation of the property of transparency of the interface: *logos* accepts only what it can accept, and honestly says when it cannot.

Connection to the conceptual model

Ankyra realizes the three roles of the family from Section 3. *External*: the proposal contract (the operator of generation, o_{abd}). *Internal*: revision (the `Revision` bookkeeping), cancellation (L1), specificity (D). *Protocol*: the journal, deterministic classification, quote grounding checks, the wave protocol, and the declaration of the fragment. Two external operators are not yet implemented: analogy (o_{anal}) and intuition (o_{int}). The system does not claim completeness: it implements what it can implement honestly, and honestly says what it cannot.

5 Discussion

We have introduced the distinction between doxa and logos, shown its formal analogues, and proposed a family of operators of the doxa–logos interaction, implemented in Ankyra. Now we must discuss the limits of this approach: what it explains, what it does not explain, and what consequences follow from it.

What the model explains

The model explains how a formal system can develop without violating its own correctness. Gödel’s and Turing’s theorems describe the limits of closed systems. Our model shows that an open system — one that interacts with doxa through an explicit interface — can expand without that expansion being a derivation within it. The expansion comes from outside. *Logos* does not produce it. It accepts, rejects, or defers it.

This is not an “overcoming” of the incompleteness theorems. It is their acceptance as a description of a boundary and the construction of an interface to what lies beyond it. Incompleteness is not removed but *relocated*: the resulting theory F' is again incomplete if it is consistent and rich enough, but the point at which the system stops has moved from the closure to the interface. We do not claim that doxa “solves” the halting problem or incompleteness. We claim only that a system that honestly acknowledges the boundary and consults an external source can continue to work where a closed system would stop.

Limits of the approach

The model does not formalize the content of doxa. We introduce this as a principled limitation, not as a temporary difficulty. Doxa is non-formalizable by nature — not because “we have not yet found a way,” but because it is not a system. It is a stream, experience, the cultural field,

embodiment, historicity. To formalize its content would be to turn it into logos, that is, to destroy what it is.

The model also does not guarantee that the set of operators we have identified is complete. We regard it as an open list. The operators of analogy and intuition, not implemented in the current version, show that the space of operators is wider than what we have described. There may exist operators we do not see. This is not a deficiency of the model but a consequence of its subject matter.

Finally, the model does not provide a criterion for the “correct” choice among competing proposals of doxa. If two operators generate incompatible hypotheses, the system records the conflict but does not resolve it automatically. The choice remains with the *Witness* — the person or external authority who bears responsibility for the choice and for whom the system exists. The model deliberately does not automate this step: no formal criterion decides between incompatible proposals of doxa.

Connection to hypercomputation

In computation theory, the notion of an oracle was introduced by Turing to describe a machine that can obtain answers to questions not solvable by its own procedure [6]. The oracle is not part of the machine, but the machine can consult it. Doxa in our model plays a role structurally close to the oracle: it does not enter the formal system, but the system can ask it questions and receive proposals.

However, there is an important difference. The oracle in computation theory gives *true* answers to questions of a certain class. Doxa does not give true answers. It gives *proposals* that may be true, false, or indeterminate. This is precisely why the interface to doxa includes classification: a proposal enters the system not as truth but as a hypothesis requiring verification. Logos does not trust doxa. It uses it.

This difference is principled. Hypercomputation in the strict sense remains a theoretical notion: it is unknown whether it is physically realizable [20]. Our model does not claim to go beyond the limits of computability, and it does not require the source to be non-computable. It describes how a system, remaining within the limits of computability, can expand through a source that is *external* to it — whether that source is computable, as an LLM is, or not.

Practical consequences

For explainable artificial intelligence, the model gives one clear principle: *an answer must not be stronger than its grounds*. If the system derived a conclusion from quotations — that is one thing. If from assumptions — another. If from a mixture — a third, and this third must be explicitly marked. The user has the right to know what the answer rests on.

For system verification, this means that verifiability does not require full formalization. It is enough that each step is traceable to a source and that each extension is recorded in the journal. Doxa remains non-formalizable, but its *trace* in the system is formal.

For the architecture of hybrid systems, this means that the division between the neural and symbolic parts is not an engineering compromise. It is a structural division reflecting the distinction between doxa and logos. The neural part is not “weaker” than the symbolic. It does what the symbolic cannot do. The symbolic part is not “stricter” than the neural. It does what the neural must not do. Each side is strong in its own way.

What remains open

The model leaves several questions open. How do operators of the different roles compose — are there laws relating external, internal and protocol steps, identities, priorities? How does one measure the effectiveness of an operator — by the depth of change in the theory, by the number of accepted proposals, by the contribution to the final answer? How does one distinguish an operator that genuinely extends the system from one that merely reformulates what is already known? These questions have no answer within the present work. They point to the direction of further research.

6 Conclusion

We have proposed an interpretation of the incompleteness theorems not as absolute limits of cognition but as limits of static systems. Human thinking is not a static system: it develops by relying on doxa — a non-formalizable source of lived experience, the cultural field, and bodily knowledge. The distinction between doxa and logos has formal analogues in modern logic, but none of them formalizes doxa as such.

Our contribution consists in introducing a family of operators of the doxa–logos interaction — a formal interface between a non-formalizable source and a formal system — and in showing that this interface is *asymmetric*: external operators supply content, internal operators decide, and protocol operators make the exchange verifiable. The content of doxa is non-formalizable, but these functions can be described formally. We have implemented this model in the Ankyra system, where a large language model plays the role of doxa and a symbolic engine plays the role of logos [19]. The principle “the LLM proposes, the engine decides” ensures transparency: nothing enters the theory without grounds, and an answer is never stronger than the grounds behind it.

We do not claim to have solved the problem of incompleteness. We claim to have shown how a system can work with this problem without attempting to overcome it. Doxa does not cancel logos — it nourishes it. And in this interaction there is not a defeat of formalization but its extension.

References

- [1] Gödel K. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I // Monatshefte für Mathematik und Physik. 1931. Bd. 38. S. 173–198.

- [2] Turing A. M. On Computable Numbers, with an Application to the Entscheidungsproblem // Proceedings of the London Mathematical Society. 1936. Ser. 2, Vol. 42. P. 230–265.
- [3] Diels H., Kranz W. Die Fragmente der Vorsokratiker. Berlin: Weidmann, 1903–1910.
- [4] Plato. Theaetetus // Plato. Collected Works: in 4 vols. Moscow: Mysl, 1990–1994. Vol. 2.
- [5] Aristotle. Nicomachean Ethics // Aristotle. Collected Works: in 4 vols. Moscow: Mysl, 1975–1983. Vol. 4.
- [6] Turing A. M. Systems of Logic Based on Ordinals // Proceedings of the London Mathematical Society. 1939. Ser. 2, Vol. 45. P. 161–228.
- [7] Hintikka J. Knowledge and Belief: An Introduction to the Logic of the Two Notions. Ithaca: Cornell University Press, 1962.
- [8] Segerberg K. The Basic Dynamic Doxastic Logic of AGM // Frontiers in Belief Revision / Ed. by M.-A. Williams, H. Rott. Dordrecht: Kluwer, 2001. P. 57–84.
- [9] Segerberg K. Some Completeness Theorems in the Dynamic Doxastic Logic of Iterated Belief Revision // The Review of Symbolic Logic. 2010. Vol. 3, No. 1. P. 1–24.
- [10] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [11] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.
- [12] Moore R. C. Semantical Considerations on Nonmonotonic Logic // Artificial Intelligence. 1985. Vol. 25, No. 1. P. 75–94.
- [13] Peirce C. S. The Beginnings of Pragmatism. St. Petersburg: Aletea, 2000.
- [14] Peirce C. S. Lectures on Pragmatism (1903) // The Collected Papers of Charles Sanders Peirce. Cambridge: Harvard University Press, 1931–1958. Vol. 5.
- [15] Niiniluoto I. Defending Abduction // Philosophy of Science. 1999. Vol. 66, No. 3. P. S436–S451.
- [16] Vinkov M. M., Fominykh I. B. Temporal Non-Monotonic Logical Systems: Interrelations and Computational Complexity // Artificial Intelligence and Decision Making. 2008. No. 4. P. 19–25.
- [17] Eiter T., Gottlob G. The Complexity of Logic-Based Abduction // Journal of the ACM. 1995. Vol. 42, No. 1. P. 3–42.
- [18] Eiter T., Gottlob G. Propositional Abduction is Σ_2^P -Complete // Information Processing Letters. 1995. Vol. 55, No. 4. P. 213–218.
- [19] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>

- [20] Cotogno P. Hypercomputation and the Physical Church-Turing Thesis // The British Journal for the Philosophy of Science. 2003. Vol. 54, No. 2. P. 181–203.
- [21] Alchourrón C. E., Gärdenfors P., Makinson D. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions // The Journal of Symbolic Logic. 1985. Vol. 50, No. 2. P. 510–530.
- [22] Kakas A. C., Kowalski R. A., Toni F. Abductive Logic Programming // Journal of Logic and Computation. 1992. Vol. 2, No. 6. P. 719–770.
- [23] Cadoli M., Schaerf M. A Survey of Complexity Results for Non-Monotonic Logics // Journal of Logic Programming. 1993. Vol. 17, No. 2–4. P. 127–160.